INFOVIZ-PROTOTYPE-EDITION

# Clustering Calculation Notes

Concise explanation of how Euclidean/KMeans and correlation-distance hierarchical clustering are calculated in the data scripts.

**Main comparison.** KMeans uses Euclidean distance in a z-scored feature space and assigns each image to the nearest centroid. Hierarchical clustering computes pairwise correlation distances between images, builds an average-linkage dendrogram, then cuts that tree into the best 2 to 5 clusters by silhouette score.

## Data Matrices

All clustering is done on a matrix where rows are images and columns are features.

```
X shape = n_images x n_features
```

| CLUSTERING TYPE | ROWS AND COLUMNS | REPO FUNCTIONS |
|---|---|---|
| **Voxel prediction clustering** | Rows are images. Columns are predicted voxel responses for one model and ROI. | `scripts/data/cluster_runner.py` `cluster_single_voxel_csv` `cluster_single_voxel_csv_hierarchical_corr` |
| **Visual clustering** | Rows are images. Columns are extracted visual features such as brightness, saturation, hue, grid statistics, edges, histograms, contrast, aspect ratio, and RGB means. | `scripts/data/cluster_runner.py` `run_visual_mode` `run_visual_hierarchical_mode` |
| **Ground-truth fMRI clustering** | Rows are images. Columns are measured fMRI voxel responses for one patient and ROI. | `scripts/data/cluster_pickle_fmri.py` `cluster_matrix` `cluster_matrix_hierarchical_corr` |

## Shared Preprocessing

The same preprocessing happens before both clustering methods.

1. Convert values to a numeric matrix.
2. Mean-impute missing values.
3. Z-score each feature column across images.

```
z = (x - mean(feature_column)) / standard_deviation(feature_column)
```

```
X = SimpleImputer(strategy="mean").fit_transform(raw_matrix)
Xz = StandardScaler().fit_transform(X)
```

For voxel prediction and ground truth, the z-scored columns are voxels. For visual clustering, the z-scored columns are image features.

## Euclidean/KMeans Method

KMeans is a flat, centroid-based clustering method. It assigns each image to the nearest cluster centroid using Euclidean distance.

```
Euclidean distance(x, c) = sqrt(sum_j((x_j − c_j)^2))
```

### Process

1. Start with the imputed, z-scored matrix `Xz`.
2. For voxel prediction and ground truth, reduce `Xz` with PCA before KMeans.
3. For visual clustering, cluster the PCA projection of the visual features.
4. Try `k = 2..5`.
5. Keep the `k` with the highest silhouette score.
6. Save each image's cluster label and distance to its assigned centroid.

### Choosing k

```
def choose_best_k(points, k_min=2, k_max=5, random_state=42):
    best = None
    for k in range(k_min, k_max + 1):
        km = KMeans(n_clusters=k, n_init=20, random_state=random_state)
        labels = km.fit_predict(points)
        score = silhouette_score(points, labels)

        if best is None or score > best["silhouette"]:
            best = {
                "k": k,
                "labels": labels,
                "silhouette": float(score),
                "model": km,
            }
    return best
```

### Voxel prediction and ground-truth KMeans

```
X = SimpleImputer(strategy="mean").fit_transform(matrix)
Xz = StandardScaler().fit_transform(X)

used_pca_dim = max(2, min(pca_dim, Xz.shape[0] - 1, Xz.shape[1]))
X_reduced = PCA(n_components=used_pca_dim, random_state=random_state).fit_transform(Xz)

best = choose_best_k(X_reduced, k_min=2, k_max=5, random_state=random_state)

distance_to_centroid = np.linalg.norm(
    X_reduced - best["model"].cluster_centers_[best["labels"]],
    axis=1,
)
```

### Visual KMeans

```
feat_df = extract_visual_feature_frame(image_dir, feature_set)
feature_cols = [c for c in feat_df.columns if c != "image_name"]
X = feat_df[feature_cols].values
Xz = StandardScaler().fit_transform(X)

pca2 = PCA(n_components=2, random_state=random_state).fit_transform(Xz)
best = choose_best_k(pca2, k_min=2, k_max=5, random_state=random_state)
```

## Correlation-Distance Hierarchical Method

This method is tree-based instead of centroid-based. It compares each pair of images by the similarity of their response pattern across voxels or features.

```
correlation_distance(x, y) = 1 − corr(x, y)
```

A distance near 0 means two images have very similar patterns. A distance near 1 means weak or no correlation. A distance approaching 2 means strongly opposite patterns.

### Process

1. Start with the imputed, z-scored matrix `Xz`.
2. Compute all pairwise correlation distances between images.
3. Build an average-linkage hierarchical tree from those distances.
4. Cut the tree into `k = 2..5` clusters.
5. Pick the cut with the highest silhouette score using the precomputed distance matrix.
6. Save the cluster labels, linkage matrix, dendrogram, and distance to cluster medoid.

### Distance matrix and linkage

```
distance_vector = clean_correlation_distance_vector(
    pdist(Xz, metric="correlation")
)
distance_matrix = squareform(distance_vector)
linkage_matrix = linkage(distance_vector, method="average")
```

### Choosing the tree cut

```
def choose_best_hierarchical_k(distance_vector, linkage_matrix, k_min=2, k_max=5):
    distance_matrix = squareform(distance_vector)
    best = None

    for k in range(k_min, k_max + 1):
        labels = fcluster(linkage_matrix, t=k, criterion="maxclust") - 1
        score = silhouette_score(distance_matrix, labels, metric="precomputed")

        if best is None or score > best["silhouette"]:
            best = {
                "k": k,
                "labels": labels,
                "silhouette": float(score),
            }
    return best
```

### Voxel prediction, visual, and ground-truth hierarchical clustering

The core calculation is the same for all three. The only difference is what the columns represent: predicted voxels, visual features, or measured fMRI voxels.

```
X = SimpleImputer(strategy="mean").fit_transform(matrix)
Xz = StandardScaler().fit_transform(X)
```

```
distance_vector = clean_correlation_distance_vector(
    pdist(Xz, metric="correlation")
)
distance_matrix = squareform(distance_vector)
linkage_matrix = linkage(distance_vector, method=linkage_method)

best = choose_best_hierarchical_k(
    distance_vector,
    linkage_matrix,
    k_min=2,
    k_max=5,
)
```

### Medoid distance

KMeans has centroids; hierarchical clustering does not. For hierarchical outputs, the representative image is the cluster medoid: the real image with the smallest total distance to other images in the same cluster.

```
medoid_distances = distance_to_cluster_medoids(
    distance_matrix,
    best["labels"],
)

result_df["distance_to_medoid"] = medoid_distances
result_df["distance_to_centroid"] = medoid_distances
```

The second column name is kept only for compatibility with existing CSV readers. For hierarchical clustering, it should be interpreted as distance to medoid by correlation distance.

## Direct Method Comparison

| PART | EUCLIDEAN/KMEANS | CORRELATION HIERARCHICAL |
|---|---|---|
| Input after preprocessing | Imputed and z-scored matrix. | Same imputed and z-scored matrix. |
| Distance | Euclidean distance to centroids. | `1 - corr(x, y)` between image pairs. |
| Cluster structure | Flat partition. | Dendrogram, then a selected flat tree cut. |
| How k is picked | Best silhouette score across `k = 2..5`. | Best silhouette score across tree cuts for `k = 2..5`. |
| Representative distance | Distance to assigned centroid. | Distance to assigned medoid. |

## Interpretation

Euclidean/KMeans is sensitive to where points sit in the z-scored/PCA space. Correlation-distance hierarchical clustering is more pattern-based: it groups images whose voxel or feature profiles vary similarly across columns, even if absolute magnitudes differ.

Therefore, changing from Euclidean KMeans to correlation-distance hierarchical clustering can change both cluster assignments and the apparent structure of the data.